

Protokoll CFS++ Entwickler Treffen am 27.07.2005

Datum: 31.08.2005

Anwesend: Kaltenbacher (ztw.), Mohr, Hauck, Bretz, Bahr, Lahmer (ztw.), Triebenbacher, Link (Protokoll)

Stand der laufenden Tätigkeiten:

- **Performance Suite:** (Ludwig und Gerhard)

Ludwig stellte das Konzept vor, dessen Grundidee die Aufspaltung des xml-Files in einen Solver und einen "restlichen" Teil ist. Wenn man nun ein Problem rechnen möchte wird ein lauffähiges xml-File durch ein Script erzeugt. Näheres zur Funktionsweise des Scripts (sh + awk) → Ludwig. Ziel ist eine Handhabung die der des cfstester ähnelt. Die mesh-Files werden in Kürze auf der lse24 liegen und werden ebenfalls von einem Script in die lokale Umgebung kopiert. Von jeder PDE wird sowohl ein 2D als auch ein 3D Problem existieren. Die Performance Suite wird in der Test Suite integriert, so dass Sie im folgenden Verzeichnis zu finden sein wird: `/CFS++_TEST/PERFORMANCESUITE`

- **IDBC Handling:** (Marcus und Andi)

CFS++/OLAS verfügt nun über zwei Mechanismen zur Behandlung von IDBCs = inhomogenen Dirichlet'schen Randbedingungen. Neben dem bisher schon vorhandenen Penalty-Ansatz besteht nun auch die Möglichkeit die Randbedingungen aus dem Gleichungssystem zu eliminieren. Hierzu waren in OLAS & CFS einige Umbauten notwendig:

- In CFS++ wurden die für die Vergabe von Gleichungsnummern zuständigen Klassen (BaseEQN, NodeEQN, ScalarNodeEQN, ScalarBlockEQN und BlockNodeEQN) so umgeschrieben, daß die Nummernvergabe dem in Abbildung 1 dargestellten Muster folgt.

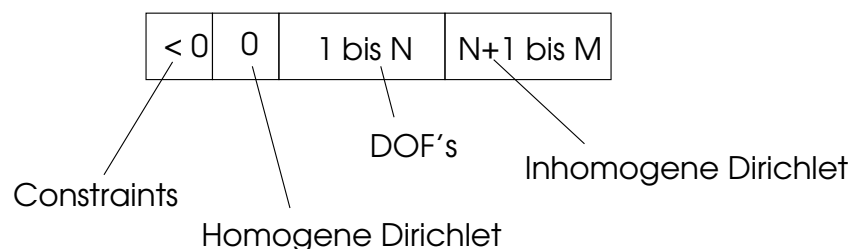


Abbildung 1: Gleichungsnummernbereiche in CFS++/OLAS

- Die Verwaltung der inhomogenen Randwerte wurde aus StandardSystem und Assemble herausgelöst und einer eigenen Klasse übergeben. Hierfür wurde die in Abbildung 2 wiedergegebene Klassenhierarchie erstellt. Die Klasse IDBC_HandlerPenalty übernimmt dabei die Implementierung des Penalty Ansatzes, während die Klasse IDBC_Handler für den Eliminationsansatz verantwortlich ist. Die Klasse IDBC_HandlerVoid ist für den Fall gedacht, daß das algebraische System zu einer PDE gehört, die über keine inhomogenen Dirichlet'schen Randwerte verfügt. Dieses Herauslösen beseitigt auch das Problem der doppelten Datenhaltung über gemeinsame Pointer zwischen StandardSystem und Assemble.

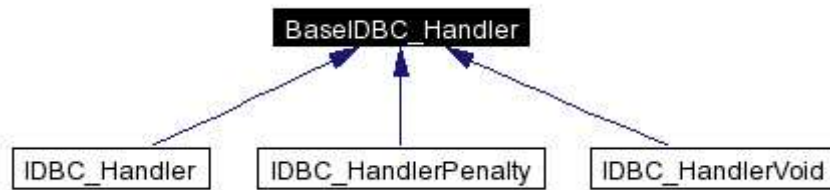


Abbildung 2: Klassenhierarchie der IDBC-Handler

- Die IDBC_Handler Klasse stellt für jede FE-Matrix eine zugehörige Hilfsmatrix auf. Aus diesen wird entsprechend dem Zusammenbau der Systemmatrix im algebraischen System, siehe ConstructEffectiveMatrix() Methode, eine Hilfssystemmatrix erstellt. Letzter wird mit dem Vektor der Dirichletwerte multipliziert und das Ergebnis von der rechten Seite abgezogen.
- Die GraphManager-Klassen mussten ebenfalls überarbeitet werden. Sie stellen die für die oben erwähnten Hilfsmatrizen benötigten Besetzungsgraphen auf. Die bisher in Assemble implementierte Umwandlung und Aussortierung von Gleichungsnummern ist in diese Klassen verlegt worden. Die Umstellung ist zunächst für GraphManagerSimple und GraphManagerStdMat durchgeführt worden. Die komplette Umstellung von GraphManagerSBMMat folgt im Zuge der Implementierung des SBM_Systems.
- Die Assemble-Klasse ist durch obige Umstellungen von einem Großteil ihrer Intelligenz, im Zusammenhang mit der Verwaltung von Randwerten und Gleichungsnummern, befreit worden. Sie konzentriert sich nun auf ihre wesentliche Aufgabe Einträge in Matrizen und Vektoren zu manipulieren. Da die Tatsache, dass es sich hierbei um eine templatisierte Klasse handelte, die aber für jeden benötigten Fall (Double, Complex, Block-Double) separat implementiert war, sehr unbefriedigend erschien, wurden die Templates gekickt und die Klassen als EntryManipulatorDouble, EntryManipulatorComplex und EntryManipulatorBlockDouble mit der gemeinsamen Basisklasse BaseEntryManipulator neu implementiert. Damit wurde auch Ludwigs Wunsch entsprochen, daß es aufgrund von Problemen mit dem gdb nicht mehr in CFS++ und OLAS eine Klasse namens Assemble geben sollte.

Es gibt noch einige kleinere Probleme mit dem Eliminationsansatz, die aber im Laufe der kommenden Wochen gelöst werden sollen.

- **SBM_System:** (Marcus) Der HiWi Michael Tesch arbeitet in OLAS an der Implementierung des SBM_Systems. Es gibt hier Fortschritte zu verzeichnen aber auch noch Probleme. Kommentar von Micheal zur Einladung auf diesem Treffen darüber zu berichten:

nothing new to report. status is that it crashes during assembly.
i think you can report that at the meeting without me.

- **Material-Klasse:** (Tom und Manfred)

Es besteht weiterhin der Wunsch nach einer deutlich mächtigeren Klasse zur Verwaltung der Materialdaten. Sie sollte z.B. die Berechnung der Materialtensoren für die Fälle 3D, plane-strain und axi übernehmen und so die (Bi-)linearformen entlasten. Das Materialfile wird auch auf xml-Format umgestellt.

Stand: In Planung und noch nicht funktionsfähig

- **Parameterübergabe CFS-OLAS:** (Marcus und Andi)

Das Problem mit der Übergabe von Parametern von CFS++ nach OLAS ist gelöst worden. Die Löserparameter werden im XML-File jetzt nicht mehr in jeder PDE einzeln, sondern in

einem separaten 'linearSystems' Block für jedes algebraische System separat spezifiziert. Damit entfällt das Problem, daß bei direkter Kopplung nicht klar war, aus welcher PDE-Sektion die Parameter zu holen waren. Momentan ist für die direkte Kopplung der Systemname 'direct' reserviert. Damit ist es im Augenblick allerdings nur möglich in der Simulation ein einzelnes direkt gekoppeltes System zu verwenden. Bei entsprechender Namensgebung sollte das Konzept aber leicht erweiterbar sein.

Stand: erledigt

- **Eliminieren der Piezo-PDE:** (Manfred und Tom) Stand: in Arbeit
- **Bereinigen der SolveStep Klassen:** (Manfred und Tom) Einbinden eines Datenobjekts, damit nicht ständig Daten weitergegeben werden müssen
Stand: in Arbeit
- **CFSP:** (Simon)
CFSP kann jetzt auch quadratische Elemente darstellen.
Des Weiteren ist die Darstellung von Vektor-Glyphen verbessert worden
Ab Herbst 2005 werden die Features von CFSP, wie Leser- und Visualisierungsroutinen in den Postprocessor SimTest vom LGDV integriert (Simon und Frank Enders)
- **Harmonic Driver:** (Marcus)
Das Sampling der Frequenzen im Intervall von startFreq bis stopFreq kann nun linear, logarithmisch und invers logarithmisch erfolgen. Details siehe [xmlReference.pdf](#)
- **GMV:** (Andi)
Es gibt jetzt eine Möglichkeit in GMV die Verschiebungen anzusehen. Verleiche hierzu die Email von Andi im Anhang 1. Außerdem gibt es jetzt die Möglichkeit Knoten-Gruppen zu selektieren
- **Schema-Definition:**(Marcus und Andi)
Neben dem bereits vorhandenen Element zur Spezifizierung des Ausgabeformates ist vor einiger Zeit ein weiteres Element hinzugefügt worden, um gmv spezifische Parameter zu übergeben. Dieses zusätzliche Element wird zur Vermeidung von Konflikten wieder gelöscht. Es wäre schön wenn Ergänzungen der Schema-Definitionen versuchen würden Inkonsistenzen zu vermeiden und auch so gestaltet sind, daß es bereits dem Parser ermöglicht wird unsinnige Parameterangaben zu detektieren. Stand: in Arbeit
- **Surface-Elemente:**(Andi)
Surface-Elemente werden jetzt auch als Region behandelt, so dass Ihnen z.B. auch ein Integrator zugewiesen werden kann
Stand: erledigt
- **Koordinatensystem**(Andi)
Andi hat CFS++ um die Möglichkeit erweitert lokale Koordinatensysteme zu definieren. Momentan kann man damit Anregungen auf unterschiedliche lokale Koordinatensysteme beziehen.
Stand: erledigt
- **Kommandozeilen Parameter**(Marcus)
Die Kommandozeilenparameter werden auch inklusive der Defaultwerte für nicht angegebene Parameter ins Info-File geschrieben.
Stand: erledigt

Neue Aufgaben/Anregungen:

- **LAPACK Methoden zur Lösung kleiner LAS's** (Tom)
Tom wird zusätzliche LAPACK-Methoden in der Matrix Klasse implementieren welche kleine linear Gleichungssystem lösen.

- **Speicherung von History-Knoten**

History Werte sollen unabhängig der Einstellung in der StoreSection für jeden Zeitschritt geschrieben werden.

- **Neues Object zum Output-Handling:** (Marcus)

Es steht weiter der Wunsch im Raum die Methoden die sich mit der Behandlung der Speicherung von Ergebnisdaten befassen zu abstrahieren und soweit als möglich aus den PDE-Klassen in eine separate Klasse zu verschieben. Ein erster Diskussionsvorschlag ist in Form des Konzepts für die HistoryManager Klasse vorhanden, siehe Anhang 2.

- **Ergebnisdaten-Handling** (Manfred)

Der mittelfristige Plan ist als Ausgabeformat nur noch das sich zur Zeit in Entstehung befindliche CFS++ eigene Format zu unterstützen. Für andere Formate wie z.B. *.rst, *.unv, *.gmV soll es Konverterprogramme geben.

Anhang 1

Date: Fri, 15 Jul 2005 16:53:52 +0200
From: Andreas Hauck <mail@andreas-hauck.com>
To: cfs-user@lse.e-technik.uni-erlangen.de
Subject: CFS/OLAS - Neuerungen

Hallo,

im CFS/OLAS gibt es ein paar Neuerungen:

1) Unter

/home/data/proj/bin/gmv-addDisp

ist nun das Programm 'gmv-addDisp' zu finden. Mit dessen Hilfe kann man mechanische Verschiebungen nachträglich auf das Gitter mappen, so dass die deformierte Struktur sichtbar wird. Das ganze funktioniert für ascii und binär, man kann entweder einen absoluten oder einen relativen Skalierungsfaktor angeben. Das alles funktioniert natürlich nur, wenn KEIN fixed-grid geschrieben wird (default).

2) Beim Schreiben eines GMV Files werden nun auch standardmäßig die benannten Knoten - also alle Randwerte und Loadknoten - mit geschrieben. Im GMV kann man diese im Nodes-Menü->select->Select on 'Node Groups' auswählen. Auf diese Weise kann man auch im Nachhinein noch feststellen, wo sich die Randwertknoten befinden.

Außerdem kann der GMV-writer nun auch bei binären Dateien alle Namen für Regionen und Ergebnisse mit 32 Zeichen statt nur mit 8 darstellen.

3) OLAS ist beim erstellen des Matrixgraphes zu Beginn viel sparsamer mit dem Speicher. Bisher konnte es nämlich passieren, dass der Matrixgraph größer als das resultierende Gleichungssystem war und das Problem deswegen nicht auf einem kleineren Rechner gerechnet werden konnte.

4) CFS schreibt nun per Default ein .profile File, in dem die verbrauchte Zeit und Speicher für die einzelnen Simulationsschritte nachzulesen sind.

Viel Spaß noch damit

Andreas

Anhang 2

The **HistoryManager** class

Suggestion for a new handling of history nodes

Dies ist ein Vorschlag für eine neue Struktur zur Behandlung von history nodes, die gleichzeitig eine Erweiterung des Konzept darstellt.

Stand der Technik

- Für das Handling der history nodes ist primär die **WriteResults**-Klasse zuständig. Sie übernimmt das Anlegen des Unterverzeichnisses und der Ausgabedateien (**InitHistory()**), sowie das eigentliche Herausschreiben der Knotenwerte (**WriteNodeHistoryTransient()**).
- Momentan wird für jeden Knoten und jede physikalische Größe eine eigene Datei angelegt und es lassen sich nur die Lösungswerte oder deren Ableitungen herrausschreiben.
- Es existiert in **WriteResults** eine Methode **WriteNodeHistoryHarmonic()**, die aber nichts tut, ausser ins trace-file zu loggen.
- Jede PDE fragt in ihrer **ReadStoreResults()** Methode das globale Parameterobjekt ab, für welche Knoten was für Größen herausgeschrieben werden müssen.
- In manchen PDEs ist das Schreiben einer nodeHistory für eine Größe an das Herausschreiben der nodeResults für diese Größe gekoppelt, was bei langen Rechnungen zu Problemen führt.
- Die zahlreichen von **WriteResults** abgeleiteten Klassen interessieren sich für die History in keinsten Weise, außer daß sie ggf. im Konstruktor **InitHistory()** aufrufen, damit überhaupt eine History geschrieben wird.

Vorschlag

Das obige Konzept scheint in meinen Augen verbesserungswürdig. Dies gilt umso mehr, als der Wunsch laut geworden ist, nicht einfach für einen Knoten die komplette History herauszuschreiben, sondern ggf. für eine Menge von Knoten in einer einzigen Daten Dinge, wie z.B. den maximalen Druckwerte über das Zeitintervall der transienten Rechnung auszugeben. Nina und ich haben uns daher das folgenden Konzept als Alternativvorschläge überlegt.

1. Die History-Sachen werden aus der **WriteResults** Klasse entfernt. Stattdessen wird eine neue Klasse **HistoryManager** angelegt.
2. Jede Standard-PDE erhält ein **HistoryManager** Objekt. Im Konstruktor der PDE füttert diese den **HistoryManager** mit folgenden Informationen:
 - Name der PDE
 - Physikalische Größen, die die PDE berechnen kann (über die zugehörigen Enumeration-Datentypen)
 - Der Tag der aktuellen Phase bei einer Multisequence-Analysis (*Noch nicht klar, die man da herankommt.*)
3. Mit den Informationen, die er von der PDE erhalten hat, kann der **HistoryManager** nun das globale Parameterobjekt befragen, um zu erfahren, für welche Knoten was für Größen herausgeschrieben werden müssen. *Dies muß also nicht mehr für jede PDE separat implementiert werden.* Außerdem kann er den **<transient>...</transient>** Parameterblock befragen, um die **saveStepIncr** Informationen etc. zu erhalten.

4. Nach Vollendung jedes Zeitschritts ruft der **TransientDriver** (über eine Durchschleifmethode der PDE) am **HistoryManager** eine Methode **ProcessNewStep()** auf. Diese Method ist nun dafür verantwortlich, zu testen, ob History-Daten herausgeschrieben werden müssen und ggf. von der PDE die hierzu benötigten Werte einzuholen.
5. Die PDE stellt eine neue Methode **GetQuantValues()** bereit, mit der man für eine zu spezifizierende physikalische Größe (z.B. **ACOU_POTENTIAL_DERIV_2**) die Werte an einer Menge von Knoten, gegeben durch einen Vektor von globalen Knotenindizes, erhält. Dies würde der PDE erlauben, die abgeleiteten Größen nur für die tatsächlich benötigten Knoten und nicht für ihr gesamtes Gebiet zu berechnen. Daneben muß ggf. noch eine weitere Methode stehen, die die PDE darauf vorbereitet, daß abgeleitete Größen benötigt werden.
6. Im XML-Parameterfile erhält jedes **nodeHistory** Element ein **format** Attribut. Der Default ist **complete**, was bedeutet, daß einfach die History komplett geschrieben wird. Ein alternativer Wert wäre z.B. **maxVal**, um sich die maximalen Werte an Knoten berechnen zu lassen. Denkbar wäre natürlich auch hier Werte wie **unv**, **gmV** o.ä. zu zulassen, um eine entsprechend formattierte Gesamthistorie in Zukunft über **nodeHistory** statt **nodeResult** zu steuern.